mra.py 的数学原理与数据流说明

2026 年 3 月 16 日

目录

1 文档范围与实现基准

本文档以仓库中的 `mra.py` 为唯一实现基准，目标是完整说明该脚本的：

- 数学建模思路；
- 各模块的张量形状变化；
- 训练、推理与异常评分的完整数据流；
- 与常见论文表述不同但在代码中真实存在的实现细节。

换言之，本文档描述的是**当前代码的真实行为**，而不仅仅是一个理想化的模型设计图。

2 符号、张量形状与超参数

2.1 核心符号

符号	形状	含义
B	标量	批大小（batch size）
S	标量	窗口长度，代码中默认 $S = 60$
N	标量	变量个数，即 CSV 列数，运行时由数据决定
$\boldsymbol{X}$	$\mathbb{R}^{B\times S\times N}$	一个批次的窗口化输入序列
$\boldsymbol{M}$	$\{0,1\}^{B\times S\times N}$	缺失掩码，1 **表示 missing**，0 **表示 observed**
$\boldsymbol{A}$	$\mathbb{R}^{N\times N}$	GraphLearner 生成的邻接矩阵
$\boldsymbol{H}^{\text{gcn}}$	$\mathbb{R}^{B\times S\times N}$	图卷积分支输出
$\boldsymbol{H}^{\text{tcn}}$	$\mathbb{R}^{B\times S\times N}$	多尺度时序卷积分支输出
$\boldsymbol{H}^{\text{time}}$	$\mathbb{R}^{B\times S\times N}$	时域分支综合表示
$\boldsymbol{H}^{\text{freq}}$	$\mathbb{R}^{B\times S\times N}$	频域分支输出
$\boldsymbol{X}^{\text{imp}}$	$\mathbb{R}^{B\times S\times N}$	门控融合后的插补表示
$\boldsymbol{Z}$	$\mathbb{R}^{B\times S\times d}$	Transformer 隐空间表示，代码中 $d = 64$
$\hat{\boldsymbol{X}}$	$\mathbb{R}^{B\times S\times N}$	最终重构输出

2.2 来自代码的固定超参数

项目	数值	说明
窗口长度 S	60	`SEQ_LEN = 60`
滑窗步长	1	`DatasetBuilder(stride=1)`
图嵌入维度	16	`GraphLearner(embed_dim=16)`
α	3.0	图嵌入缩放因子
TCN 卷积核	$\{3,5,9\}$	主模型中实际使用的三种尺度
TCN 模式	non-causal	主模型中 `causal=False`
Transformer 隐维 d	64	`d_model=64`
注意力头数	4	`nhead=4`

项目	数值	说明
Transformer 层数	2	`num_layers=2`
FFN 隐维	128	`dim_feedforward=128`
Dropout	0.1	Transformer 编码层内置 dropout
优化器学习率	10^{-3}	Adam 初始学习率
调度器 factor	0.5	`ReduceLROnPlateau` 衰减倍数
调度器 patience	10	10 个 epoch 不改进则降学习率
训练轮数	3	代码中当前写死为 `for epoch in range(3)`
批大小	32	训练和推理默认均为 32
频域损失权重	0.1	`freq_weight` 默认值
图熵正则权重	0.1	`sparsity_weight` 默认值

3 整体流程总览

3.1 训练期总流程

训练时，一个窗口样本从数据到损失的大流程可以写成：

$$(\boldsymbol{X}, \boldsymbol{M}) \xrightarrow{\text{随机再遮挡}} (\boldsymbol{X}^{\text{in}}, \boldsymbol{M}^{\text{in}}, \boldsymbol{T}) \tag{1}$$

$$\xrightarrow{\text{AGF-ADNet}} (\hat{\boldsymbol{X}}, \boldsymbol{A}, \boldsymbol{X}^{\text{imp}}) \tag{2}$$

$$\xrightarrow{\text{dual_domain_loss}} \mathcal{L}_{\text{rec}} + 0.1\,\mathcal{L}_{\text{freq}} + 0.1\,\mathcal{L}_{\text{ent}} \tag{3}$$

其中：

- $\boldsymbol{X}$ 是原始窗口；
- $\boldsymbol{M}$ 是原始缺失掩码；
- $\boldsymbol{M}^{\text{in}}$ 是加入随机遮挡后的输入掩码；
- $\boldsymbol{T}$ 是本批次真正参与监督的目标位置掩码；
- $\boldsymbol{A}$ 是学习到的图结构；
- $\hat{\boldsymbol{X}}$ 是最终重构输出。

3.2 推理期总流程

推理时没有随机再遮挡，因此流程更简单：

$$(\boldsymbol{X}, \boldsymbol{M}) \xrightarrow{\text{apply_missing_mask}} \boldsymbol{X}^{\text{in}} \xrightarrow{\text{AGF-ADNet}} \hat{\boldsymbol{X}} \xrightarrow{\text{masked MSE}} \text{Score} \tag{4}$$

测试阈值并不是通过验证集调参得到，而是直接由训练集分数的均值与标准差估计：

$$\tau = \mu_{\text{train}} + \sigma_{\text{train}}. \tag{5}$$

4 数据准备与窗口构造

4.1 目录读取与掩码定义

`DatasetBuilder.load_dir()` 从指定目录下读取一组 CSV 文件，并按文件名排序后顺序拼接。设单个原始时间序列为

$$\boldsymbol{D} \in \mathbb{R}^{T\times N}, \tag{6}$$

其中 T 为时间长度，N 为变量数。

代码对缺失值使用如下掩码约定：

$$M_{t,n} = \begin{cases} 1, & D_{t,n} \text{ 为缺失值} \\ 0, & D_{t,n} \text{ 为观测值} \end{cases} \tag{7}$$

这个定义与很多论文中“1 表示 observed”正好相反，因此后续阅读代码时必须持续记住该语义。

4.2 标准化

标准化前，代码先把缺失值用 0 临时填充：

$$\bar{D}_{t,n} = \begin{cases} 0, & D_{t,n} \text{ 缺失} \\ D_{t,n}, & \text{否则} \end{cases} \tag{8}$$

然后仅在训练集上拟合 `StandardScaler`：

$$\mu_n = \frac{1}{T_{\text{train}}} \sum_{t=1}^{T_{\text{train}}} \bar{D}_{t,n}, \tag{9}$$

$$\sigma_n^2 = \frac{1}{T_{\text{train}}} \sum_{t=1}^{T_{\text{train}}} (\bar{D}_{t,n} - \mu_n)^2, \tag{10}$$

并对训练集和测试集统一变换：

$$D_{t,n}^{\text{scaled}} = \frac{\bar{D}_{t,n} - \mu_n}{\sigma_n}. \tag{11}$$

实现含义 这里的标准化对象不是“带 NaN 的真实数据”，而是“先用 0 填过 NaN 的数组”。因此，缺失位置在标准化之后仍然会带有一个数值占位符；真正让缺失位置退出模型输入的是后面的 `apply_missing_mask()`。

4.3 滑动窗口

设标准化后的序列记为 $\boldsymbol{D}^{\text{scaled}} \in \mathbb{R}^{T\times N}$，窗口长度为 S，步长为 $\delta = 1$。代码以时间索引 i 为窗口终点，构造第 i 个窗口：

$$\boldsymbol{X}^{(i)} = \begin{cases} [\underbrace{\boldsymbol{d}_0, \ldots, \boldsymbol{d}_0}_{S-i-1}, \boldsymbol{d}_0, \boldsymbol{d}_1, \ldots, \boldsymbol{d}_i], & i < S \\ [\boldsymbol{d}_{i-S+1}, \ldots, \boldsymbol{d}_i], & i \geq S \end{cases} \tag{12}$$

其中 $\boldsymbol{d}_t \in \mathbb{R}^N$ 为第 t 个时间步的向量。

掩码窗口以完全相同的方式构造：

$$\boldsymbol{M}^{(i)} = \begin{cases} [\underbrace{\boldsymbol{m}_0, \ldots, \boldsymbol{m}_0}_{S-i-1}, \boldsymbol{m}_0, \boldsymbol{m}_1, \ldots, \boldsymbol{m}_i], & i < S \\ [\boldsymbol{m}_{i-S+1}, \ldots, \boldsymbol{m}_i], & i \geq S \end{cases} \tag{13}$$

最终一个批次输入模型的张量形状就是：

$$\boldsymbol{X}, \boldsymbol{M} \in \mathbb{R}^{B \times S \times N}. \tag{14}$$

5 模型主体：AGF-ADNet

5.1 前向传播总式

对于一个输入窗口批次，`AGF_ADNet.forward(x, mask)` 的主流程可以抽象为：

$$\boldsymbol{A} = \text{GraphLearner}() \tag{15}$$

$$\boldsymbol{H}^{\text{time}} = \text{TimeBranch}(\boldsymbol{X}^{\text{in}}, \boldsymbol{A}) \tag{16}$$

$$\boldsymbol{H}^{\text{freq}} = \text{FreqBranch}(\boldsymbol{X}^{\text{in}}) \tag{17}$$

$$\boldsymbol{X}^{\text{imp}} = \text{GatedFusion}(\boldsymbol{H}^{\text{time}}, \boldsymbol{H}^{\text{freq}}) \tag{18}$$

$$\hat{\boldsymbol{X}} = \text{TransformerHead}(\boldsymbol{X}^{\text{filled}}) \tag{19}$$

其中 $\boldsymbol{X}^{\text{filled}}$ 的**理想形式**应当是“观测位置保留原值，缺失位置使用插补值”，但当前代码的真实行为与理想公式存在差异，后文会专门说明。

5.2 图结构学习模块：GraphLearner

5.2.1 参数化

GraphLearner 使用两组可学习节点嵌入：

$$\boldsymbol{E}_1, \boldsymbol{E}_2 \in \mathbb{R}^{N \times d_e}, \qquad d_e = 16. \tag{20}$$

它们先经过缩放与双曲正切非线性：

$$\boldsymbol{M}_1 = \tanh(\alpha \boldsymbol{E}_1), \tag{21}$$

$$\boldsymbol{M}_2 = \tanh(\alpha \boldsymbol{E}_2), \tag{22}$$

其中 $\alpha = 3.0$。

5.2.2 邻接矩阵计算

原始邻接分数为：

$$\boldsymbol{A}^{\text{raw}} = \text{ReLU}(\boldsymbol{M}_1 \boldsymbol{M}_2^\top). \tag{23}$$

随后进行逐行归一化：

$$A_{ij} = \frac{A_{ij}^{\text{raw}}}{\sum_{k=1}^{N} A_{ik}^{\text{raw}} + \varepsilon}, \qquad \varepsilon = 10^{-8}. \tag{24}$$

为什么不是 softmax 代码刻意避免使用 softmax，而采用 ReLU 后再做行归一化。原因是 softmax 会让所有边权都严格大于 0，图结构难以变得真正尖锐；当前实现后面还会对邻接矩阵施加熵约束，因此需要允许某些边直接变为 0。

5.3 时域分支

5.3.1 GCNLayer

输入先扩展一个特征维：

$$\boldsymbol{X}^{\text{in}} \in \mathbb{R}^{B\times S\times N} \longrightarrow \boldsymbol{X}^{\text{in,4D}} \in \mathbb{R}^{B\times S\times N\times 1}. \tag{25}$$

图卷积层先做线性映射：

$$\tilde{\boldsymbol{X}}_{b,s,n,:} = \boldsymbol{W}\boldsymbol{X}^{\text{in,4D}}_{b,s,n,:} + \boldsymbol{b}, \tag{26}$$

然后按邻接矩阵在节点维聚合：

$$H^{\text{gcn}}_{b,s,n,f} = \sum_{m=1}^{N} A_{n,m}\,\tilde{X}_{b,s,m,f}. \tag{27}$$

在代码中该操作由

$$\texttt{torch.einsum("nm,bsmd->bsnd", adj, x)} \tag{28}$$

实现。

线性层的输入输出维度都是 1，因此最终再 squeeze 回到

$$\boldsymbol{H}^{\text{gcn}} \in \mathbb{R}^{B\times S\times N}, \tag{29}$$

并执行 LayerNorm：

$$\boldsymbol{H}^{\text{gcn}} \leftarrow \text{LayerNorm}(\boldsymbol{H}^{\text{gcn}}). \tag{30}$$

5.3.2 MultiScaleTCN

输入先转置到通道优先格式：

$$\boldsymbol{X}^{\text{in}} \in \mathbb{R}^{B\times S\times N} \longrightarrow \boldsymbol{X}^{\text{tcn}} \in \mathbb{R}^{B\times N\times S}. \tag{31}$$

对每个卷积核尺寸 $k \in \{3,5,9\}$，代码使用一层 depthwise Conv1D：

$$\boldsymbol{Y}^{(k)} = \text{Conv1D}_{k,\ groups=N}(\text{Pad}(\boldsymbol{X}^{\text{tcn}})). \tag{32}$$

因为在主模型中 `causal=False`，所以 padding 采用对称方式。对于奇数卷积核，有：

$$p_{\text{left}} = \frac{k-1}{2}, \qquad p_{\text{right}} = \frac{k-1}{2}. \tag{33}$$

每个节点都是独立卷积，不发生跨节点通道混合，因此第 n 个节点、第 s 个时间步的输出可写为：

$$Y^{(k)}_{b,n,s} = \sum_{i=0}^{k-1} w^{(k)}_{n,i}\,\tilde{X}^{\text{tcn}}_{b,n,s+i-p_{\text{left}}} + b^{(k)}_{n}. \tag{34}$$

三种尺度输出堆叠为：

$$\boldsymbol{Y}^{\text{stack}} \in \mathbb{R}^{B\times N\times S\times 3}, \tag{35}$$

再经过一个线性层 $3 \to 1$ 融合：

$$\boldsymbol{H}^{\text{tcn}} = \text{Linear}_{3\to1}(\boldsymbol{Y}^{\text{stack}}). \tag{36}$$

最后转回 (B, S, N)，并与 GCN 分支相加：

$$\boldsymbol{H}^{\text{time}} = \boldsymbol{H}^{\text{gcn}} + \boldsymbol{H}^{\text{tcn}}. \tag{37}$$

5.4 频域分支：FrequencyImputer

5.4.1 时域到频域

输入先转为 (B, N, S)：

$$\boldsymbol{X}^{\text{freq-in}} = \text{permute}(\boldsymbol{X}^{\text{in}}) \in \mathbb{R}^{B\times N\times S}. \tag{38}$$

然后沿时间维做实数快速傅里叶变换：

$$\boldsymbol{X}_f = \text{rFFT}(\boldsymbol{X}^{\text{freq-in}}) \in \mathbb{C}^{B\times N\times F}, \qquad F = \left\lfloor \frac{S}{2} \right\rfloor + 1. \tag{39}$$

代码中还计算了：

$$\boldsymbol{R} = \Re(\boldsymbol{X}_f), \tag{40}$$

$$\boldsymbol{I} = \Im(\boldsymbol{X}_f), \tag{41}$$

$$|\boldsymbol{X}_f| = \text{magnitude}, \tag{42}$$

$$\angle \boldsymbol{X}_f = \text{phase}. \tag{43}$$

实现注记 `magnitude` 与 `phase` 在当前代码中被计算出来，但后续并未参与损失或前向增强过程。真正进入神经网络的是实部与虚部的拼接特征。

5.4.2 频率注意力与增强

构造频域实值特征：

$$\boldsymbol{Z}_f = [\boldsymbol{R}; \boldsymbol{I}] \in \mathbb{R}^{B\times N\times 2F}. \tag{44}$$

注意力网络输出频率权重：

$$\boldsymbol{\alpha} = \sigma\big(\boldsymbol{W}_2^{\text{att}}\, \phi\big(\boldsymbol{W}_1^{\text{att}} \boldsymbol{Z}_f + \boldsymbol{b}_1^{\text{att}}\big) + \boldsymbol{b}_2^{\text{att}}\big) \in [0,1]^{B\times N\times F}, \tag{45}$$

其中 $\phi(\cdot)$ 表示 ReLU。

增强网络输出：

$$\hat{\boldsymbol{Z}}_f = \boldsymbol{W}_2^{\text{enh}}\, \phi\big(\boldsymbol{W}_1^{\text{enh}} \boldsymbol{Z}_f + \boldsymbol{b}_1^{\text{enh}}\big) + \boldsymbol{b}_2^{\text{enh}} \in \mathbb{R}^{B\times N\times 2F}. \tag{46}$$

将其拆成实部增强量与虚部增强量：

$$\hat{\boldsymbol{R}} = \hat{\boldsymbol{Z}}_f[:, :, 1:F], \tag{47}$$

$$\hat{\boldsymbol{I}} = \hat{\boldsymbol{Z}}_f[:, :, F+1:2F]. \tag{48}$$

更准确地说，若按 0-based 编号，代码切片实际是：

$$\hat{\boldsymbol{R}} = \hat{\boldsymbol{Z}}_f[:, :, 0:F], \tag{49}$$

$$\hat{\boldsymbol{I}} = \hat{\boldsymbol{Z}}_f[:, :, F:2F]. \tag{50}$$

频率注意力逐元素作用于增强量：

$$\boldsymbol{R}^{\text{att}} = \hat{\boldsymbol{R}} \odot \boldsymbol{\alpha}, \tag{51}$$

$$\boldsymbol{I}^{\text{att}} = \hat{\boldsymbol{I}} \odot \boldsymbol{\alpha}. \tag{52}$$

随后以残差方式修正原始频谱：

$$\boldsymbol{X}_f^{\text{enh}} = \boldsymbol{X}_f + \left(\boldsymbol{R}^{\text{att}} + j\,\boldsymbol{I}^{\text{att}}\right). \tag{53}$$

5.4.3 频域回到时域

最后做逆实傅里叶变换并转回原排列：

$$\boldsymbol{H}^{\text{freq,raw}} = \text{irFFT}(\boldsymbol{X}_f^{\text{enh}}) \in \mathbb{R}^{B\times N\times S}, \tag{54}$$

$$\boldsymbol{H}^{\text{freq}} = \text{LayerNorm}(\text{permute}^{-1}(\boldsymbol{H}^{\text{freq,raw}})) \in \mathbb{R}^{B\times S\times N}. \tag{55}$$

5.5 门控融合：GatedFusion

将时域和频域分支分别转到 (B, N, S)，然后拼接：

$$\boldsymbol{C} = [\boldsymbol{H}^{\text{time}}; \boldsymbol{H}^{\text{freq}}] \in \mathbb{R}^{B\times 2N\times S}. \tag{56}$$

门控网络结构是：

$$\boldsymbol{Z}_g = \sigma\Big(\text{Conv1D}_{1\times 1}\left(\phi(\text{Conv1D}_{k=3}(\boldsymbol{C}))\right)\Big) \in [0,1]^{B\times N\times S}. \tag{57}$$

转回 (B, S, N) 后，逐元素融合：

$$\boldsymbol{X}^{\text{imp}} = \text{LayerNorm}\left(\boldsymbol{Z}_g \odot \boldsymbol{H}^{\text{time}} + (1-\boldsymbol{Z}_g)\odot \boldsymbol{H}^{\text{freq}}\right). \tag{58}$$

解释 当某个位置的门值接近 1 时，模型更相信时域分支；当门值接近 0 时，更相信频域分支。

5.6 Transformer 重构头

5.6.1 输入投影

理论上，融合后的插补序列应形成 $\boldsymbol{X}^{\text{filled}}$，再送入 Transformer。代码中的投影为：

$$\boldsymbol{Z}_0 = \boldsymbol{X}^{\text{filled}}\boldsymbol{W}_{\text{in}} + \boldsymbol{b}_{\text{in}} + \boldsymbol{P}, \tag{59}$$

其中：

$$\boldsymbol{W}_{\text{in}} \in \mathbb{R}^{N\times d}, \qquad \boldsymbol{P} \in \mathbb{R}^{1\times S\times d}, \qquad d = 64. \tag{60}$$

5.6.2 多头自注意力编码器

对每一层 Transformer 编码器，采用标准 post-norm 结构。设第 l 层输入为 $\boldsymbol{Z}_{l-1}$，则：

$$\boldsymbol{Q}_h = \boldsymbol{Z}_{l-1}\boldsymbol{W}_h^Q, \tag{61}$$

$$\boldsymbol{K}_h = \boldsymbol{Z}_{l-1}\boldsymbol{W}_h^K, \tag{62}$$

$$\boldsymbol{V}_h = \boldsymbol{Z}_{l-1}\boldsymbol{W}_h^V, \tag{63}$$

第 h 个头的注意力为：

$$\mathrm{head}_h = \mathrm{softmax}\left(\frac{\boldsymbol{Q}_h \boldsymbol{K}_h^\top}{\sqrt{d_h}}\right)\boldsymbol{V}_h. \tag{64}$$

多头拼接后：

$$\mathrm{MHA}(\boldsymbol{Z}_{l-1}) = \mathrm{Concat}(\mathrm{head}_1, \ldots, \mathrm{head}_4)\boldsymbol{W}^O. \tag{65}$$

结合残差与层归一化：

$$\boldsymbol{U}_l = \mathrm{LayerNorm}(\boldsymbol{Z}_{l-1} + \mathrm{MHA}(\boldsymbol{Z}_{l-1})), \tag{66}$$

$$\boldsymbol{Z}_l = \mathrm{LayerNorm}(\boldsymbol{U}_l + \mathrm{FFN}(\boldsymbol{U}_l)). \tag{67}$$

代码中共有两层此类编码器，即 $l = 1, 2$。

5.6.3 输出投影

最终通过线性层恢复到变量维度：

$$\hat{\boldsymbol{X}} = \boldsymbol{Z}_2 \boldsymbol{W}_{\mathrm{out}} + \boldsymbol{b}_{\mathrm{out}}, \qquad \boldsymbol{W}_{\mathrm{out}} \in \mathbb{R}^{d \times N}. \tag{68}$$

6 关于 $\boldsymbol{X}^{\mathrm{filled}}$ 的实现细节

6.1 代码中的真实写法

在 `AGF_ADNet.forward()` 中，与插补有关的代码逻辑是：

$$\boldsymbol{O} = 1 - \boldsymbol{M}, \tag{69}$$

$$\boldsymbol{O} = \boldsymbol{M}, \tag{70}$$

$$\boldsymbol{X}^{\mathrm{filled}} = \boldsymbol{X}^{\mathrm{in}} \odot \boldsymbol{O} + \boldsymbol{X}^{\mathrm{imp}} \odot \boldsymbol{M}. \tag{71}$$

由于第二行直接覆盖了第一行，因此代码真实执行的是：

$$\boldsymbol{X}^{\mathrm{filled}} = \boldsymbol{X}^{\mathrm{in}} \odot \boldsymbol{M} + \boldsymbol{X}^{\mathrm{imp}} \odot \boldsymbol{M}. \tag{72}$$

6.2 进一步化简

又因为进入模型前已经做过

$$\boldsymbol{X}^{\mathrm{in}} = \boldsymbol{X} \odot (1 - \boldsymbol{M}), \tag{73}$$

所以有

$$\boldsymbol{X}^{\mathrm{in}} \odot \boldsymbol{M} = \boldsymbol{0}. \tag{74}$$

因此当前代码下 Transformer 实际接收的是：

$$\boldsymbol{X}^{\mathrm{filled}} = \boldsymbol{X}^{\mathrm{imp}} \odot \boldsymbol{M}. \tag{75}$$

含义 这表示：

- 被标记为 missing 的位置进入 Transformer 时保留插补值；
- 被标记为 observed 的位置进入 Transformer 时为 0。

与常见插补逻辑的差异 更常见、也更直觉的写法通常是：

$$\boldsymbol{X}^{\text{filled,ideal}} = \boldsymbol{X}^{\text{in}} \odot (1 - \boldsymbol{M}) + \boldsymbol{X}^{\text{imp}} \odot \boldsymbol{M}, \tag{76}$$

即“观测位置保留原值，缺失位置填补预测值”。但本文档在其余部分仍以**代码真实行为**为准。

7 损失函数：dual_domain_loss

7.1 监督目标掩码

训练时，损失并不是在全部位置上计算，而是只在 `target_mask` 指定的位置上监督：

$$T_{b,s,n} \in \{0, 1\}. \tag{77}$$

在代码里，T 通常来自“从原本可观测位置中再随机遮掉 10%”这一策略；若某个 batch 恰好一个位置都没抽中，则退化为对全部原始可观测位置监督。

7.2 时域重构损失

重构误差项为：

$$\mathcal{L}_{\text{rec}} = \frac{\sum_{b,s,n} \left(\hat{X}_{b,s,n} - X_{b,s,n}\right)^2 T_{b,s,n}}{\max\left(\sum_{b,s,n} T_{b,s,n}, 1\right)}. \tag{78}$$

7.3 有效样本筛选

频域损失并非对所有样本都算，而是先基于原始缺失掩码计算每个样本的观测比例：

$$r_b = \frac{1}{SN} \sum_{s=1}^{S} \sum_{n=1}^{N} (1 - M_{b,s,n}). \tag{79}$$

仅当

$$r_b > 0.5 \tag{80}$$

时，该样本才进入频域损失计算集合：

$$\mathcal{B}_{\text{valid}} = \{b \mid r_b > 0.5\}. \tag{81}$$

7.4 频域损失

代码先构造一个频域目标：

$$\boldsymbol{X}^{\text{freq-target}} = \text{stopgrad}(\hat{\boldsymbol{X}}) \odot (1 - \boldsymbol{T}) + \boldsymbol{X} \odot \boldsymbol{T}. \tag{82}$$

这意味着：

- 在有监督的位置，用真实值 $\boldsymbol{X}$；
- 在无监督的位置，用当前预测值的 detached 版本；
- 因而频域损失不会把无监督区域的误差梯度反传回去。

对有效样本，比较重构与目标的幅值谱：

$$\hat{\boldsymbol{F}}_b = \mathrm{rFFT}(\hat{\boldsymbol{X}}_b^\top), \tag{83}$$

$$\boldsymbol{F}_b^\star = \mathrm{rFFT}((\boldsymbol{X}_b^{\text{freq-target}})^\top), \tag{84}$$

$$\mathcal{L}_{\text{freq}} = \frac{1}{|\mathcal{B}_{\text{valid}}|} \sum_{b\in\mathcal{B}_{\text{valid}}} \frac{1}{NF} \sum_{n=1}^{N} \sum_{f=1}^{F} \left(\left|\hat{F}_{b,n,f}\right| - \left|F^\star_{b,n,f}\right| \right)^2. \tag{85}$$

若没有任何有效样本，则代码直接令：

$$\mathcal{L}_{\text{freq}} = 0. \tag{86}$$

7.5 图熵正则

尽管函数参数名叫 `sparsity_weight`，当前代码使用的并不是 L_1 稀疏正则，而是邻接矩阵的**行熵**：

$$\mathcal{L}_{\text{ent}} = -\frac{1}{N} \sum_{i=1}^{N} \sum_{j=1}^{N} A_{ij} \log(A_{ij} + \varepsilon). \tag{87}$$

由于 $\boldsymbol{A}$ 已经逐行归一化，这个量越小，说明每一行分布越尖锐，图结构越接近“只关注少数关键邻居”。

7.6 总损失

最终损失为：

$$\mathcal{L} = \mathcal{L}_{\text{rec}} + 0.1\,\mathcal{L}_{\text{freq}} + 0.1\,\mathcal{L}_{\text{ent}}. \tag{88}$$

8 训练流程的数学化描述

8.1 随机再遮挡策略

设原始缺失掩码为 $\boldsymbol{M}$，原始可观测掩码为：

$$\boldsymbol{O} = 1 - \boldsymbol{M}. \tag{89}$$

代码在可观测位置上额外随机采样一个遮挡指示：

$$D_{b,s,n} \sim \mathrm{Bernoulli}(0.1), \qquad \boldsymbol{R} = \boldsymbol{D} \odot \boldsymbol{O}. \tag{90}$$

因此真正被随机遮掉的位置是原本可观测且被采样命中的位置。监督掩码定义为：

$$\boldsymbol{T} = \boldsymbol{R}. \tag{91}$$

若本批次中 $\sum \boldsymbol{T} = 0$，代码退化为：

$$\boldsymbol{T} = \boldsymbol{O}. \tag{92}$$

8.2 构造输入掩码与输入张量

输入掩码是对原始缺失掩码的拷贝，并把随机遮挡位置也设为缺失：

$$\boldsymbol{M}^{\text{in}} = \boldsymbol{M}, \qquad M^{\text{in}}_{b,s,n} = 1 \text{ if } T_{b,s,n} = 1. \tag{93}$$

模型真实接收的输入是：

$$\boldsymbol{X}^{\text{in}} = \boldsymbol{X} \odot (1 - \boldsymbol{M}^{\text{in}}). \tag{94}$$

一个重要细节 `dual_domain_loss()` 接收到的 `missing_mask` 仍然是原始 $\boldsymbol{M}$，不是 $\boldsymbol{M}^{\text{in}}$。也就是说：

- 训练输入会看到“原始缺失 + 人工再遮挡”；
- 但频域有效样本筛选依据的是“原始缺失比例”。

8.3 优化

每个 batch 的优化步骤是：

1. 前向传播得到 $\hat{\boldsymbol{X}}, \boldsymbol{A}$；
2. 计算总损失 $\mathcal{L}$；
3. 反向传播；
4. 进行梯度裁剪：
$$\|\nabla\|_2 \leftarrow \min(\|\nabla\|_2, 1.0); \tag{95}$$
5. 由 Adam 更新参数。

每个 epoch 结束后，脚本使用平均训练损失作为 `ReduceLROnPlateau` 的输入，并保存训练损失最低时的模型参数快照。

实现事实 这里没有独立验证集；所谓“best model”是按训练集平均损失选出来的。

9 推理、异常分数与分类评价

9.1 异常分数

推理阶段，模型对每个窗口输出重构结果 $\hat{\boldsymbol{X}}$。异常分数定义为原始可观测位置上的平均平方误差：

$$\text{Score}_b = \frac{\sum_{s=1}^{S}\sum_{n=1}^{N}(\hat{X}_{b,s,n} - X_{b,s,n})^2(1 - M_{b,s,n})}{\sum_{s=1}^{S}\sum_{n=1}^{N}(1 - M_{b,s,n}) + 10^{-8}}. \tag{96}$$

这里分母的 `clamp_min(1e-8)` 用来避免极端情况下除零。

9.2 阈值估计

训练完成后，脚本先在**训练窗口**上计算一遍异常分数：

$$\{\text{Score}_1^{\text{train}}, \ldots, \text{Score}_K^{\text{train}}\}. \tag{97}$$

阈值定义为：

$$\tau = \text{mean}(\text{Score}^{\text{train}}) + \text{std}(\text{Score}^{\text{train}}). \tag{98}$$

实现事实 这不是三倍标准差规则，而是一倍标准差规则，即 `mean + std`。

9.3 测试标签构造与评价

`build_test_labels(num_scores)` 直接把测试集后半段标为异常：

$$y_i = \begin{cases} 0, & i < \lfloor K/2 \rfloor \\ 1, & i \geq \lfloor K/2 \rfloor \end{cases} \tag{99}$$

于是预测标签为：

$$\hat{y}_i = \mathbb{I}[\mathrm{Score}_i > \tau]. \tag{100}$$

随后计算准确率、精确率、召回率和 F1：

$$\mathrm{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}, \tag{101}$$

$$\mathrm{Precision} = \frac{TP}{TP + FP}, \tag{102}$$

$$\mathrm{Recall} = \frac{TP}{TP + FN}, \tag{103}$$

$$F_1 = \frac{2 \cdot \mathrm{Precision} \cdot \mathrm{Recall}}{\mathrm{Precision} + \mathrm{Recall}}. \tag{104}$$

实现事实 这种标签构造方式隐含了一个强假设：测试窗口在时间顺序上“前半正常、后半异常”。

10 逐层张量数据流表

10.1 模型前向阶段

阶段	张量表达式	形状
输入窗口	$\boldsymbol{X}$	$B \times S \times N$
输入掩码	$\boldsymbol{M}^{\mathrm{in}}$	$B \times S \times N$
缺失置零	$\boldsymbol{X}^{\mathrm{in}} = \boldsymbol{X} \odot (1 - \boldsymbol{M}^{\mathrm{in}})$	$B \times S \times N$
图学习输出	$\boldsymbol{A}$	$N \times N$
GCN 输入扩维	$\boldsymbol{X}^{\mathrm{in}}.\mathrm{unsqueeze}(-1)$	$B \times S \times N \times 1$
GCN 输出	$\boldsymbol{H}^{\mathrm{gcn}}$	$B \times S \times N$
TCN 转置输入	$\mathrm{permute}(\boldsymbol{X}^{\mathrm{in}})$	$B \times N \times S$
三尺度堆叠	$\boldsymbol{Y}^{\mathrm{stack}}$	$B \times N \times S \times 3$
TCN 输出	$\boldsymbol{H}^{\mathrm{tcn}}$	$B \times S \times N$
时域融合	$\boldsymbol{H}^{\mathrm{time}}$	$B \times S \times N$
频域 FFT 输入	$\mathrm{permute}(\boldsymbol{X}^{\mathrm{in}})$	$B \times N \times S$
频谱张量	$\boldsymbol{X}_f$	$B \times N \times F$（复数）
频域输出	$\boldsymbol{H}^{\mathrm{freq}}$	$B \times S \times N$
门控拼接张量	$\boldsymbol{C}$	$B \times 2N \times S$
门控图	$\boldsymbol{Z}_g$	$B \times S \times N$
插补结果	$\boldsymbol{X}^{\mathrm{imp}}$	$B \times S \times N$
Transformer 输入	$\boldsymbol{X}^{\mathrm{filled}}$	$B \times S \times N$
输入投影后	$\boldsymbol{Z}_0$	$B \times S \times 64$

阶段	张量表达式	形状
Transformer 输出	$\boldsymbol{Z}_2$	$B \times S \times 64$
最终重构	$\hat{\boldsymbol{X}}$	$B \times S \times N$

10.2 训练外层流程

阶段	数据流说明
CSV 读取	读取 `./data/train/train_*.csv` 与 `./data/test/test_*.csv`，生成原始数组与缺失掩码。
标准化	在训练集上拟合 `StandardScaler`，对训练与测试都做同一组变换。
窗口化	构造长度 60 的滑动窗口，序列开头不足长度时用首行复制填充。
训练输入生成	在原始可观测位置中随机遮掉 10%，形成 `m_input` 与 `x_input`。
模型前向	`AGF_ADNet(x_input, m_input)` 输出重构序列、图结构与插补表示。
损失计算	用原始窗口 `x`、原始缺失掩码 `m`、目标掩码 `target_mask` 计算双域损失。
最优模型保存	按平均训练损失选取最佳参数快照。
训练阈值估计	对训练窗口跑一遍异常分数，阈值取 `mean + std`。
测试集评价	对测试窗口计算分数，再和阈值比较，输出准确率、精确率、召回率、F1。

11 实现层面的关键观察

1. **掩码语义是反直觉的**。在整个脚本里，1 表示 missing，0 表示 observed。
2. **标准化发生在零填充之后**。缺失值在标准化前先被置为 0，再参与均值方差计算。
3. **频域分支真正使用的是实部/虚部**。幅值与相位虽然被计算，但没有直接参与增强或损失。
4. **图正则不是 L_1，而是熵**。这与很多图学习论文中的“稀疏正则”写法不同。
5. **阈值是 1σ 规则**。代码实现为训练分数均值加一个标准差，而非三倍标准差。
6. **测试标签生成依赖顺序假设**。后半段强制视为异常，是一种数据集特定约定，不是通用异常标注方式。
7. **当前 Transformer 输入只保留掩码位置的插补值**。这是由 `observed_mask` 覆盖造成的真实行为。
8. **最佳模型按训练损失选取**。没有单独验证集，也没有早停机制。

12 总结

`mra.py` 实现的是一个以**重构误差驱动异常检测**为核心思想的模型，主要由以下几层机制叠加而成：

1. 用图学习模块自适应建模变量间依赖；
2. 用 GCN 与多尺度 TCN 提取时域结构；
3. 用 FFT、频率注意力和频域残差增强提取频谱结构；
4. 用门控机制在时域与频域之间做动态融合；
5. 用 Transformer 做全局时序重构；
6. 用时域误差、频域谱误差和图熵约束联合训练；
7. 用训练集分数的均值加标准差作为异常阈值。

如果把它抽象成一句话，那么这份代码的数学本质可以概括为：

> 先把多变量时间序列在**图结构**、**时域局部模式**、**频域全局模式**三个视角下进行表征，再通过融合与 Transformer 重构建立“正常行为模型”，最后用**掩码加权的重构误差**判断异常。

但在实际部署或复现实验时，仍应特别注意本文档指出的实现细节，尤其是掩码语义、阈值公式以及 `X_filled` 的当前代码行为。